Operations Runbook

Day-to-day operational tasks for the Madagascar Freshwater Fish Conservation Platform (MFFCP). Focused on the **staging environment** (Hetzner-hosted) and the **local development** equivalents.

For first-time staging bootstrap (provisioning the VM, installing Docker, wiring the GitHub Actions deploy, initial seeding), see `deploy/staging/README.md`. This document covers the *recurring* tasks that come up after that bootstrap is done.

---

0. Quick Reference

Thing	Value
Staging hostname	`mffcp-staging`
Staging IP	`46.224.196.197`
Staging SSH user	`deploy`
Public site (Vercel)	`https://malagasyfishes.org`
Backend API (staging)	`https://api.malagasyfishes.org`
Revalidate webhook	`https://malagasyfishes.org/api/revalidate`
Repo on staging	`/home/deploy/madagascarfish`
Docker Compose dir on staging	`/home/deploy/madagascarfish/deploy/staging`
Compose `.env` location	`/home/deploy/madagascarfish/deploy/staging/.env`
Seed CSVs on host	`/home/deploy/madagascarfish/data/seed/*.csv`
Seed CSVs **inside container**	`/data/seed/*.csv`
Django app inside container	`/app`
Backups dir	`/home/deploy/backups/`

The compose file is at `deploy/staging/docker-compose.yml`. All `docker compose` commands **must be run from that directory** — otherwise compose can't find the `.env` file and errors with `env file /home/deploy/madagascarfish/.env not found`.

---

1. Connecting to Staging

```
ssh deploy@46.224.196.197
```

First-time connection will prompt to trust the host key. If you see a host-key mismatch warning later, *stop* — someone may be MITM'ing, or the server was rebuilt. Confirm out-of-band before removing the old key.

Once on the box, the two directories you care about are:

```
cd /home/deploy/madagascarfish           # for git operations
cd /home/deploy/madagascarfish/deploy/staging  # for docker compose operations
```

You'll toggle between the two.

---

2. Pulling the Latest Code

When to use: before running any seed, migration, or deploy command that depends on new code or new data files. The GitHub Actions deploy workflow does this automatically on merge to `main`, but you may need to pull manually when the workflow is gated (e.g. CI was red) or when you want to run a seed immediately after a merge rather than waiting for the next deploy.

```
cd /home/deploy/madagascarfish
git fetch origin
git log --oneline origin/main -3     # sanity-check the top commits
git pull --ff-only                    # fast-forward only; refuses if diverged
```

Why `--ff-only`: staging should never have local commits. If fast-forward fails, something is wrong — investigate before resetting.

---

3. Rebuilding / Restarting Containers

When to use: after a backend code change (Python deps, Django settings, Dockerfile), after an `.env` change, or if a container is in a bad state.

All from `/home/deploy/madagascarfish/deploy/staging`:

```
# Rebuild images (needed when backend/Dockerfile or requirements.txt changed)
docker compose build

# Start or restart in the background
docker compose up -d

# Restart just one service without rebuilding
docker compose restart web
docker compose restart worker
docker compose restart beat

# Tail logs
docker compose logs -f web
docker compose logs -f --tail=200 worker
```

Note: CSV-only changes under `data/seed/` do **not** require a rebuild. `data/` is bind-mounted into the container at `/data`, so `git pull` is enough — the new files are visible on next command.

---

4. Running Database Migrations

When to use: after merging a PR that adds a Django migration.

```
cd /home/deploy/madagascarfish/deploy/staging
docker compose exec -T web python manage.py migrate
```

The `-T` flag disables TTY allocation — required for non-interactive shells (scripts, CI, one-liners over SSH). You can omit it when running the command interactively from a logged-in terminal; both work.

To check migration state without applying anything:

```
docker compose exec -T web python manage.py showmigrations --plan
```

---

5. Seeding Data

This is the most common recurring task. The seed layer has three commands, invoked from `/home/deploy/madagascarfish/deploy/staging`.

5.1 Full seed (everything at once)

```
docker compose exec -T web python manage.py seed_all
```

Runs in order:

1. `load_reference_layers` — watersheds + protected areas (GeoJSON)
2. `seed_species` — species catalog from CSV
3. `seed_localities` — locality records from CSV

All three are **idempotent** — safe to re-run. Existing rows get `update_or_create` treatment keyed on scientific name (species) or `(species, location_key, locality_type)` (localities).

5.2 Partial seed

When you know only part of the data changed, skip the expensive steps:

```
# Species CSV and/or localities CSV changed, but reference layers didn't
docker compose exec -T web python manage.py seed_all --skip-reference

# Only localities changed
docker compose exec -T web python manage.py seed_all --skip-reference --skip-species

# Only reference layers changed (rare)
docker compose exec -T web python manage.py seed_all --skip-species --skip-
localities
```

`load_reference_layers` is the slowest step (spatial simplification of watershed and protected-area polygons), so `--skip-reference` is the common optimization for seed-data PRs that only touch the CSVs.

5.3 Direct invocation with explicit paths

Useful for one-off testing (e.g. a branch-specific CSV) or to target a file outside the default path.

```
docker compose exec -T web python manage.py seed_species \
  --csv /data/seed/madagascar_freshwater_fish_seed.csv

docker compose exec -T web python manage.py seed_localities \
  --csv /data/seed/madagascar_freshwater_fish_localities_seed.csv
```

Critical: the `--csv` path must be the **in-container path** (`/data/seed/...`), not the host path (`/home/deploy/madagascarfish/data/seed/...` or `data/seed/...`). The `./data` directory on the host is bind-mounted to `/data` inside the container; Django only sees the in-container path.

If you pass the host path you get:

```
CommandError: CSV not found: data/seed/madagascar_freshwater_fish_localities_seed.csv
```

5.4 Dry-run before committing changes to prod data

`seed_localities` supports `--dry-run` (wrapped in a transaction that's rolled back):

```
docker compose exec -T web python manage.py seed_localities \
  --csv /data/seed/madagascar_freshwater_fish_localities_seed.csv --dry-run
```

5.5 Verifying a seed run

After any seed run, verify row counts landed as expected:

```
docker compose exec -T web python manage.py shell -c \
  "from species.models import Species, SpeciesLocality; \
   print('species:', Species.objects.count(), \
         'localities:', SpeciesLocality.objects.count())"
```

Expected values live in `docs/planning/specs/data-preparation-guide.md` .

5.6 Interpreting the output

A healthy `seed_all` run looks like:

```
== load_reference_layers ==
watersheds: 0 created, 82 updated, 32 parented (vertices 51606 -> 42524)
protected areas: 0 created, 55 updated (vertices 60490 -> 8161)
== seed_species ==
created: 13
updated: 143
skipped: 0
== seed_localities ==
created: 119
updated: 587
skipped: 0
no drainage basin matched: 54
seed_all complete
```

- **`created`** — new rows added this run.

- **`updated`** — existing rows that were touched (idempotent re-write).
- **`skipped`** — rows that failed validation or couldn't match a species. **If this is non-zero, read the lines printed to stderr afterwards** — they name the CSV line number and the reason. Common causes:
  - `species not found` — locality references a scientific name not in the species catalog. Fix: add the species to the species CSV first, or correct the locality's scientific name.
  - `presence_status='X' not in [...]` — enum value doesn't match `SpeciesLocality.PresenceStatus`. Fix: normalize the CSV value.
  - `water_body_type='Y' not in [...]` — same pattern for water body type.
- **`no drainage basin matched`** — informational, not an error. The row is inserted with `drainage_basin=NULL`. Common for points outside the watershed coverage (e.g. coastal, or slightly offshore GPS error). Re-match later by improving the watershed layer.

---

6. The Seed CSV Pipeline

The localities seed has **two** CSVs by design:

File	Role
`data/seed/madagascar_freshwater_fish_localities_seed_master.csv`	**Source of truth.** Unfiltered. Accumulates every locality we've collected, regardless of MVP scope.
`data/seed/madagascar_freshwater_fish_localities_seed.csv`	**Loaded by `seed_localities`.** Derived from master by applying documented filters (drop genus-only names, drop out-of-bounds coordinates).

Rule: never hand-edit the seed CSV as the source of truth. Edit the master, then regenerate the seed. The filters are documented in `docs/planning/specs/data-preparation-guide.md` § "Master File and Filter Rules".

A third file, `data/seed/new_localities_from_local_files.csv`, is a **triage backlog** — locality records that arrived from local sources but aren't yet resolved into the catalog (e.g. non-endemic peripheral species, hybrids). It's not loaded by any command.

Workflow for a new batch of locality data:

1. Normalize columns to match the master header.
2. Diff against the master (species + lat/lng + source citation) to identify novel rows.
3. For each novel row, decide: merge as-is, remap to an existing species, defer to triage file, or drop.
4. Append to master.
5. Regenerate the seed by re-applying filters (script lives in this repo's history; the filters are simple enough to inline).
6. Update the row-count line in `data-preparation-guide.md`.
7. Commit on a `data/<description>` branch, PR, merge.
8. Run the seed on staging.

---

7. One-off Shell / Inspection

Open a Django shell on the container:

```
cd /home/deploy/madagascarfish/deploy/staging
docker compose exec web python manage.py shell
```

(No `-T` — interactive shell needs a TTY.)

Open a Postgres shell:

```
docker compose exec db psql -U mffcp mffcp
```

Peek at a running process without attaching:

```
docker compose ps
docker compose top web
```

---

8. Creating / Managing Admin Users

Initial superuser (done at bootstrap):

```
cd /home/deploy/madagascarfish/deploy/staging
docker compose exec web python manage.py createsuperuser
```

Promote an existing user to staff/superuser later:

```
docker compose exec web python manage.py shell -c \
  "from django.contrib.auth import get_user_model; \
   U = get_user_model(); u = U.objects.get(email='someone@example.org'); \
   u.is_staff = True; u.is_superuser = True; u.save(); print('done')"
```

Admin lives at `https://<staging-url>/admin/`.

---

9. Backups and Restore

A cron job on the staging VM writes a daily `pg_dump`:

```
0 3 * * *  cd /home/deploy/madagascarfish/deploy/staging && \
           docker compose exec -T db pg_dump -U mffcp mffcp | gzip \
           > /home/deploy/backups/mffcp-$(date +%F).sql.gz
0 4 * * 0  find /home/deploy/backups -name 'mffcp-*.sql.gz' -mtime +14 -delete
```

Run a manual backup before anything risky (migrations that drop columns, bulk data rewrites):

```
cd /home/deploy/madagascarfish/deploy/staging
docker compose exec -T db pg_dump -U mffcp mffcp | gzip \
```

```
  > /home/deploy/backups/mffcp-manual-$(date +%F-%H%M).sql.gz
```

Restore from a dump (destructive — drops and recreates the DB):

```
cd /home/deploy/madagascarfish/deploy/staging
gunzip -c /home/deploy/backups/mffcp-2026-04-20.sql.gz | \
  docker compose exec -T db psql -U mffcp -d postgres -c \
  "DROP DATABASE IF EXISTS mffcp; CREATE DATABASE mffcp;"
gunzip -c /home/deploy/backups/mffcp-2026-04-20.sql.gz | \
  docker compose exec -T db psql -U mffcp mffcp
```

Pull a backup locally for inspection:

```
# From your laptop
scp deploy@46.224.196.197:/home/deploy/backups/mffcp-2026-04-20.sql.gz ./
```

---

10. Log Inspection

```
cd /home/deploy/madagascarfish/deploy/staging

# Last N lines, follow
docker compose logs --tail=500 -f web

# Just errors and above (naive grep)
docker compose logs web | grep -iE "error|traceback|critical"

# Across all services
docker compose logs --tail=200
```

Container stdout/stderr is the primary log sink — we don't ship to an external aggregator yet.

---

11. Shared Secrets (Vercel ↔ Django)

Two shared secrets bridge the Next.js frontend (Vercel, `malagasyfishes.org`) and the Django backend (Hetzner staging). Both **must match exactly** across the two environments, and both default to blank — blank disables the feature with the safer posture.

Setting	Django env	Vercel env	What it does
Next.js revalidate webhook	`NEXT_REVALIDATE_URL` + `NEXT_REVALIDATE_SECRET`	`REVALIDATE_SECRET`	Admin save → public page refresh
Coordinator dashboard auth	`COORDINATOR_API_TOKEN`	`COORDINATOR_API_TOKEN`	SSR fetch of Tier 3+ panels

Plus three frontend-only NextAuth secrets that don't have Django counterparts — see §11.3.

11.1 Next.js cache revalidation (ISR webhook)

The public site uses ISR with `revalidate = 3600`, so a species/genus edit in Django admin takes up to an hour to appear publicly — unless the backend fires the revalidate webhook on save. `SpeciesAdmin`, `GenusAdmin`, and `SiteMapAssetAdmin` all call it automatically; the admin also exposes a manual **"Revalidate public pages"** action.

Wiring:

- Frontend route: `POST https://malagasyfishes.org/api/revalidate` (checks `REVALIDATE_SECRET` env; 401 on mismatch).
- Backend sender: `backend/species/admin_revalidate.py::_post_revalidate()` (reads `NEXT_REVALIDATE_URL` + `NEXT_REVALIDATE_SECRET`).
- URL must point at the canonical (non-redirecting) domain — `malagasyfishes.org`, not `www.malagasyfishes.org` (the `www` variant 308s; POSTs don't follow redirects reliably).

Configuring / rotating:

```
# 1. Generate a secret
openssl rand -hex 32

# 2. Vercel env:
#    REVALIDATE_SECRET = <the secret>
# Redeploy Vercel.

# 3. Staging backend:
ssh deploy@46.224.196.197
cd /home/deploy/madagascarfish/deploy/staging
nano .env
# Add:
#   NEXT_REVALIDATE_URL=https://malagasyfishes.org/api/revalidate
#   NEXT_REVALIDATE_SECRET=<same secret>
docker compose up -d --force-recreate web
```

Verifying: edit a species in admin and save. The banner should say "Revalidated N path(s)." If it says "Revalidate is not configured...", the backend env is unset or the container wasn't recreated after editing `.env`. HTTP 401 means the secrets don't match.

11.2 Coordinator dashboard service token

`/dashboard/coordinator` (Gate 3 Ex-situ Coordinator view) renders four Tier 3+ endpoints in the Next.js server runtime. Session cookies don't cross the Vercel ↔ Hetzner origins, so the page uses a shared secret sent in `Authorization: Bearer <token>`. The token is server-only and never reaches the browser.

Wiring:

- Frontend (Vercel env): `COORDINATOR_API_TOKEN`
- Backend (`deploy/staging/.env`): `COORDINATOR_API_TOKEN`
- Blank on the backend disables the bypass entirely — only real Tier 3+ sessions work. Blank on Vercel means the page will render all-panels in the "Unable to load..." fallback state.

Configuring / rotating:

```
# 1. Generate a secret
openssl rand -hex 32

# 2. Vercel env:
#    COORDINATOR_API_TOKEN = <the secret>
# Redeploy Vercel.

# 3. Staging backend:
ssh deploy@46.224.196.197
cd /home/deploy/madagascarfish/deploy/staging
nano .env
# Add: COORDINATOR_API_TOKEN=<same secret>
docker compose up -d --force-recreate web
```

Verifying:

```
# With token → 200
curl -s -o /dev/null -w "%{http_code}\n" \
  -H "Authorization: Bearer <token>" \
  https://api.malagasyfishes.org/api/v1/coordinator-dashboard/stale-census/

# Without token → 403
curl -s -o /dev/null -w "%{http_code}\n" \
  https://api.malagasyfishes.org/api/v1/coordinator-dashboard/stale-census/
```

Open `/dashboard/coordinator` — all four panels (coverage gap, studbook, sex-ratio, stale census) should render data. Every panel in fallback state ⇒ Vercel env is unset or doesn't match the backend value. Check the Vercel deployment's Functions log for an upstream 403.

11.3 NextAuth secrets (Gate 11)

NextAuth lives **only on the frontend**. There are no Django-side counterparts to keep in sync — all three of these secrets are Vercel-only.

Setting	Where	What it does
`NEXTAUTH_SECRET`	Vercel env (per-environment scope)	JWT signing key. Different value per env. Required.
`NEXTAUTH_URL`	Vercel env (per-environment scope)	Canonical frontend URL. Required.
`NEXT_PUBLIC_FEATURE_AUTH`	Vercel env (per-environment scope)	`"true"` to expose auth nav + middleware redirects. Default unset / `"false"`.

Configuring / rotating `NEXTAUTH_SECRET`:

```
# 1. Generate a fresh secret on your laptop
openssl rand -hex 32  # one per environment — never reuse across envs

# 2. Vercel — Project Settings → Environment Variables:
```

```
#    NEXTAUTH_SECRET = <hex>, scope to the matching environment (Production,
Preview, Development)
#    NEXTAUTH_URL    = https://malagasyfishes.org (Production)
#                      http://localhost:3000      (Development; lives in
frontend/.env.local)
# 3. Save, redeploy.
```

Rotation cadence: annual for `NEXTAUTH_SECRET` under normal operation. On suspected leak: rotate immediately. Rotating invalidates every existing session (JWTs signed with the old secret can't decode); acceptable cost.

Vercel preview deploys stay anonymous (architecture §9). The session cookie is scoped to `.malagasyfishes.org` (or whatever the Vercel project is configured for); `*.vercel.app` preview origins can't read it. This is intentional — UAT for authenticated flows happens on the production domain, not preview URLs. Don't try to make preview authenticated; that path leads to cross-origin cookie hacks that bypass `Secure; SameSite=Lax`.

Verifying: browse `/login` while signed out, sign in, reload — the header should show "Account / Sign out" instead of "Sign in / Sign up". In browser DevTools, the session cookie should be `__Secure-next-auth.session-token` with `HttpOnly`, `Secure`, `SameSite=Lax`. No NextAuth warnings should appear in Vercel's function logs.

Preview env vars. Add `NEXTAUTH_SECRET` (a fresh `openssl rand -hex 32` value, distinct from prod) and `NEXTAUTH_URL` (literal placeholder like `https://preview.placeholder`, OR `https://${VERCEL_URL}` to use Vercel's dynamic preview hostname) to the **Preview** environment scope. These silence `[next-auth][error][NO_SECRET]` 500s on `/api/auth/session` from the client-side `useSession()` poll. Real auth flows still don't complete on preview because the prod cookie domain (`.malagasyfishes.org`) doesn't match `*.vercel.app` — which is the architecture's intent.

11.4 Verifying the coordinator-dashboard auth path (cookie-domain check)

`getServerDrfToken()` reads the NextAuth session cookie via `next/headers`, decodes the JWT with `NEXTAUTH_SECRET`, and pulls out the user's DRF token. If anything in that chain fails — wrong cookie name, mismatched domain, secret rotation that didn't redeploy — the helper returns `undefined` and the coordinator-dashboard SSR **silently falls back** to `COORDINATOR_API_TOKEN`.

The dashboard still renders. Tier 3+ data still shows. But every signed-in user is anonymously bypassing tier — the audit trail no longer reflects who fetched what, and the architectural goal of Gate 11 ("no service tokens mailed around") is silently undermined.

This runbook walks through the verification we ran on 2026-04-28 to confirm the path is healthy. Re-run after any change touching `authOptions` in `frontend/lib/auth.ts`, after rotating `NEXTAUTH_SECRET`, or any time you suspect Tier 3+ users might be on the service-token path without realizing it.

The diagnostic mechanism. `coordinatorHeaders()` in `frontend/lib/coordinatorDashboard.ts` emits one of three server-side console logs on every panel fetch:

- `[coordinator-auth] path=session` — DRF token from the user's session. This is the goal.
- `[coordinator-auth] path=service-token-fallback` — `COORDINATOR_API_TOKEN` from Vercel env. Indicates session resolution failed.
- `[coordinator-auth] path=none` — neither was available. Endpoints will 403 and panels render the "token not configured" banner.

Vercel surfaces server-side `console.log` in Project → Logs. The token value is never logged.

The check:

1. Have a real Tier 3+ user account on production. If you don't have one, sign up via `/signup`, verify via the email link, then in Django admin on `api.malagasyfishes.org/admin/` set `access_tier = 3` on the user.
2. Sign in to `https://malagasyfishes.org/login` with that account.
3. Confirm the session cookie in DevTools → Application → Cookies → `https://malagasyfishes.org`. Expect `__Secure-next-auth.session-token` with `HttpOnly`, `Secure`, `SameSite=Lax`, `Domain` either blank (defaults to host) or matching `malagasyfishes.org`.
4. Navigate to `https://malagasyfishes.org/dashboard/coordinator`. Wait for all panels to render.
5. In Vercel → Logs (filter to last few minutes, environment = production), grep for `[coordinator-auth]`. You should see multiple `path=session` lines — one per panel fetch — clustered around the request timestamp.

Pass: every recent `[coordinator-auth]` log shows `path=session`.

Fail: any `path=service-token-fallback` or `path=none` for a request made by a signed-in user. The fix is in `frontend/lib/auth.ts` `authOptions`:

```
cookies: {
  sessionToken: {
    name: process.env.NODE_ENV === "production"
      ? "__Secure-next-auth.session-token"
      : "next-auth.session-token",
    options: {
      httpOnly: true,
      sameSite: "lax",
      path: "/",
      secure: process.env.NODE_ENV === "production",
      domain: process.env.NODE_ENV === "production"
        ? ".malagasyfishes.org"
        : undefined,
    },
  },
},
```

The leading-dot domain (`.malagasyfishes.org`) makes the cookie readable from both apex and `www`. Don't speculatively apply this — verify the failure mode first; the wrong domain configuration breaks things in different ways.

Cross-check via `COORDINATOR_API_TOKEN` blank. Indirect proof if you want a second signal: temporarily blank `COORDINATOR_API_TOKEN` in Vercel prod env, redeploy, sign in as Tier 3, hit the dashboard. If panels still render → session path works. If they show "Unable to load..." → service token was masking a session-resolution failure. **Restore the token after the test** — anonymous SSR (page-render before any user signs in) and the public coordination summary tiles depend on it.

---

12. Common Failure Modes and Fixes

12.1 `env file /home/deploy/madagascarfish/.env not found`

You're running `docker compose` from the wrong directory. The `.env` lives in `deploy/staging/`, not the repo root.

Fix: `cd /home/deploy/madagascarfish/deploy/staging` first.

12.2 `CSV not found: data/seed/...`

You passed a host-style path to a command running inside the container.

Fix: use the container path `--csv /data/seed/...` (leading slash, `/data` not `data`).

12.3 `species not found: 'Foo bar'` during `seed_localities`

The locality CSV references a scientific name that isn't in the species catalog yet.

Fix: either

- add the species to `madagascar_freshwater_fish_seed.csv` first, re-run `seed_species`, then re-run `seed_localities`; or
- correct the name in the locality CSV (often a typo or case mismatch — e.g. `Bedotia sp. 'Manombo'` vs. existing `Bedotia sp. 'manombo'`).

12.4 `presence_status='X' not in [...]` or `water_body_type='Y' not in [...]`

CSV has an enum value that doesn't match the model choices.

Fix: normalize the value in the CSV. Allowed values are documented in `data-preparation-guide.md` § "Column Reference" and enforced by `SpeciesLocality.PresenceStatus` / `SpeciesLocality.WaterBodyType` in `backend/species/models.py`.

12.5 `git pull --ff-only` refuses

Staging has local commits, which it shouldn't. Find out what happened before doing anything destructive:

```
cd /home/deploy/madagascarfish
git log --oneline main origin/main --not $(git merge-base main origin/main)
git status
```

If the local commits are legitimate work someone did by hand on the server, preserve them on a branch before resetting. Don't `git reset --hard` reflexively.

12.6 Staging is behind `main` on GitHub even after a merge

The GitHub Actions deploy (`workflow_run`) runs only if the CI workflow succeeded. If CI was red, deploy is skipped silently.

Check: the **Actions** tab on GitHub for both CI and Deploy runs on the merge commit.

Fix manually: SSH in and `git pull` yourself (§2), then run whatever seed/migrate/restart is needed.

---

13. Local Development Equivalents

The commands are near-identical to staging — just run them from the repo root on your laptop, and use the root `docker-compose.yml` (no `deploy/staging/` prefix):

```
cd ~/Repos/MadagascarFish

# Bring up the stack
docker compose up -d

# Run migrations
docker compose exec web python manage.py migrate

# Seed everything
docker compose exec web python manage.py seed_all

# Seed just the changed parts
docker compose exec web python manage.py seed_all --skip-reference

# Direct invocation (note: container path, same as staging)
docker compose exec web python manage.py seed_localities \
  --csv /data/seed/madagascar_freshwater_fish_localities_seed.csv

# Verify counts
docker compose exec web python manage.py shell -c \
  "from species.models import Species, SpeciesLocality; \
   print(Species.objects.count(), SpeciesLocality.objects.count())"

# Django shell / Postgres shell
docker compose exec web python manage.py shell
docker compose exec db psql -U mffcp mffcp

# Reset local DB entirely (destructive — loses all local data)
docker compose down -v
docker compose up -d
docker compose exec web python manage.py migrate
docker compose exec web python manage.py seed_all
```

The local compose does **not** need a `cd deploy/staging` dance — the `.env` lives in the repo root for local, and in `deploy/staging/` for staging. That's the one structural difference.

13.1 Running the Next.js frontend locally

The Django API on `:8000` is only half the stack — the public site is the Next.js app in `frontend/`, run outside Docker.

```
cd frontend

# One-time: create the env file the Next.js runtime reads. Without this,
# every page renders the "temporarily unavailable" empty state because
# NEXT_PUBLIC_API_URL is undefined and all fetches fail fast.
cp .env.example .env.local
# .env.local now contains:
```

```
#   NEXT_PUBLIC_API_URL=http://localhost:8000
#   NEXT_REVALIDATE_SECONDS=3600

pnpm install          # first time only
pnpm dev              # starts on http://localhost:3000
```

Gotchas:

- **Port 3000 in use** → Next falls back to 3001. Check the dev server log for the actual port. Usually means a previous `pnpm dev` is still running.
- **Every page says "temporarily unavailable"** → `.env.local` is missing or `NEXT_PUBLIC_API_URL` is wrong, or the Django stack isn't running. Verify with `curl http://localhost:8000/api/v1/species/` — should 200.
- **CSS or tokens look stale after editing `globals.css` / `tailwind.config.ts`** → Next usually hot-reloads but occasionally needs a full server restart. Ctrl-C and `pnpm dev` again.

Full-stack smoke test sequence from cold:

```
docker compose up -d                # Django + Postgres + Redis on :8000
cd frontend && pnpm dev             # Next on :3000
open http://localhost:3000/         # home
open http://localhost:3000/species/ # directory
```

---

14. Standard Post-Merge Flow for a Data/Seed PR

The happy-path sequence after a CSV-only PR (e.g. #61, #62) merges to `main`:

```
# 1. SSH in
ssh deploy@46.224.196.197

# 2. Pull latest
cd /home/deploy/madagascarfish
git pull --ff-only

# 3. Reseed the changed data (skip expensive reference layer step)
cd deploy/staging
docker compose exec -T web python manage.py seed_all --skip-reference

# 4. Verify counts match the data-preparation-guide
docker compose exec -T web python manage.py shell -c \
  "from species.models import Species, SpeciesLocality; \
   print(Species.objects.count(), SpeciesLocality.objects.count())"

# 5. If skipped > 0, read the errors and open a follow-up fix PR.
```

For a code/migration PR, the flow is:

```
ssh deploy@46.224.196.197
cd /home/deploy/madagascarfish && git pull --ff-only
cd deploy/staging
```

```
docker compose build                        # if Dockerfile or requirements changed
docker compose up -d                        # pick up new image
docker compose exec -T web python manage.py migrate
docker compose restart worker beat          # if Celery tasks changed
```

The GitHub Actions deploy does all of this automatically on green CI — you only run it by hand when the automation didn't run (red CI) or when iterating.